

Практическая работа Java

2. Настройка базы данных

```
1  spring:
2    datasource:
3      url: jdbc:h2:mem:taskdb
4      driverClassName: org.h2.Driver
5      username: sa
6      password: password
7    h2:
8      console:
9        enabled: true
10   jpa:
11     hibernate:
12       ddl-auto: update
```

3. Создание модели задачи

Добавляем зависимости

```
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

```

demo > src > main > java > com > example > demo > model > J Task.java
1  package com.example.demo.model;
2
3  import jakarta.persistence.Entity;
4  import jakarta.persistence.GeneratedValue;
5  import jakarta.persistence.GenerationType;
6  import jakarta.persistence.Id;
7  import lombok.Data;
8  import lombok.NoArgsConstructor;
9  import lombok.AllArgsConstructor;
10
11 import java.time.LocalDateTime;
12
13 @Entity
14 @Data
15 @NoArgsConstructor
16 @AllArgsConstructor
17 public class Task {
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private Long id;
21     Ctrl+L to chat, Ctrl+K to generate
22     private String title;
23     private String description;
24     private String status;
25     private LocalDateTime createdAt;
26 }

```

4. Создание репозитория для задач

```

demo > src > main > java > com > example > demo > repository > J TaskRepository.java
1  package com.example.demo.repository;
2
3  import com.example.demo.model.Task;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.stereotype.Repository;
6
7  @Repository
8  public interface TaskRepository extends JpaRepository<Task, Long> {
9  }

```

5. Создание REST контроллера

Создадим класс для обработки ошибок:

```
package com.example.demo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Теперь создадим сам контроллер:

```
demo > src > main > java > com > example > demo > controller > TaskController.java
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.LocalDateTime;
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/tasks")
14 public class TaskController {
15
16     @Autowired
17     private TaskRepository taskRepository;
18
19     @GetMapping
20     public List<Task> getAllTasks() {
21         return taskRepository.findAll();
22     }
23
24     @PostMapping
25     public Task createTask(@RequestBody Task task) {
26         task.setCreatedAt(LocalDateTime.now());
27         return taskRepository.save(task);
28     }
29
30     @PutMapping("/{id}")
31     public Task updateTask(@PathVariable Long id, @RequestBody Task updatedTask) {
32         return taskRepository.findById(id)
33             .map(task -> {
34                 task.setTitle(updatedTask.getTitle());
35                 task.setDescription(updatedTask.getDescription());
36                 task.setStatus(updatedTask.getStatus());
37                 return taskRepository.save(task);
38             })
39             .orElseThrow(() -> new ResourceNotFoundException("Задача с id " + id + " не найдена"));
40     }
41
42     @DeleteMapping("/{id}")
43     public void deleteTask(@PathVariable Long id) {
44         taskRepository.deleteById(id);
45     }
46 }
```

6. Добавление безопасности

добавим зависимость в pom.xml

```
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
</dependencies>
```

обновляем application.yml

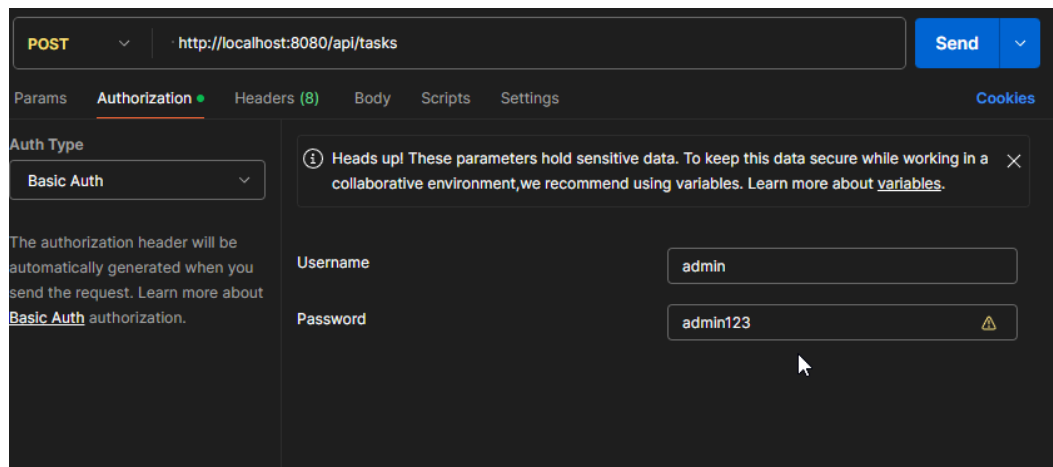
```
demo > src > main > resources > ! application.yml
1  spring:
2    datasource:
3      url: jdbc:h2:mem:taskdb
4      driverClassName: org.h2.Driver
5      username: sa
6      password: password
7    h2:
8      console:
9        enabled: true
10   jpa:
11     hibernate:
12       ddl-auto: update
13
14   security:
15     user:
16       name: admin
17       password: admin123
```

Создадим конфигурацию безопасности

```
demo > src > main > java > com > example > demo > config > SecurityConfig.java
1  package com.example.demo.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6  import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
7  import org.springframework.security.web.SecurityFilterChain;
8
9  @Configuration
10 @EnableWebSecurity
11 public class SecurityConfig {
12
13     @Bean
14     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
15         http
16             .csrf(csrf -> csrf.disable())
17             .authorizeHttpRequests(auth -> auth
18                 .requestMatchers("/api/tasks/**").authenticated()
19                 .anyRequest().authenticated()
20             )
21             .httpBasic();
22
23         return http.build();
24     }
25 }
```

7. Тестирование приложения

1. Создание задачи (POST)



The screenshot shows the 'Authorization' tab of a REST client. The 'Auth Type' is set to 'Basic Auth'. A warning message states: 'Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about variables.' The 'Username' field contains 'admin' and the 'Password' field contains 'admin123'.

POST ▼ http://localhost:8080/api/tasks Send ▼

Params **Authorization** Headers (8) Body Scripts Settings Cookies

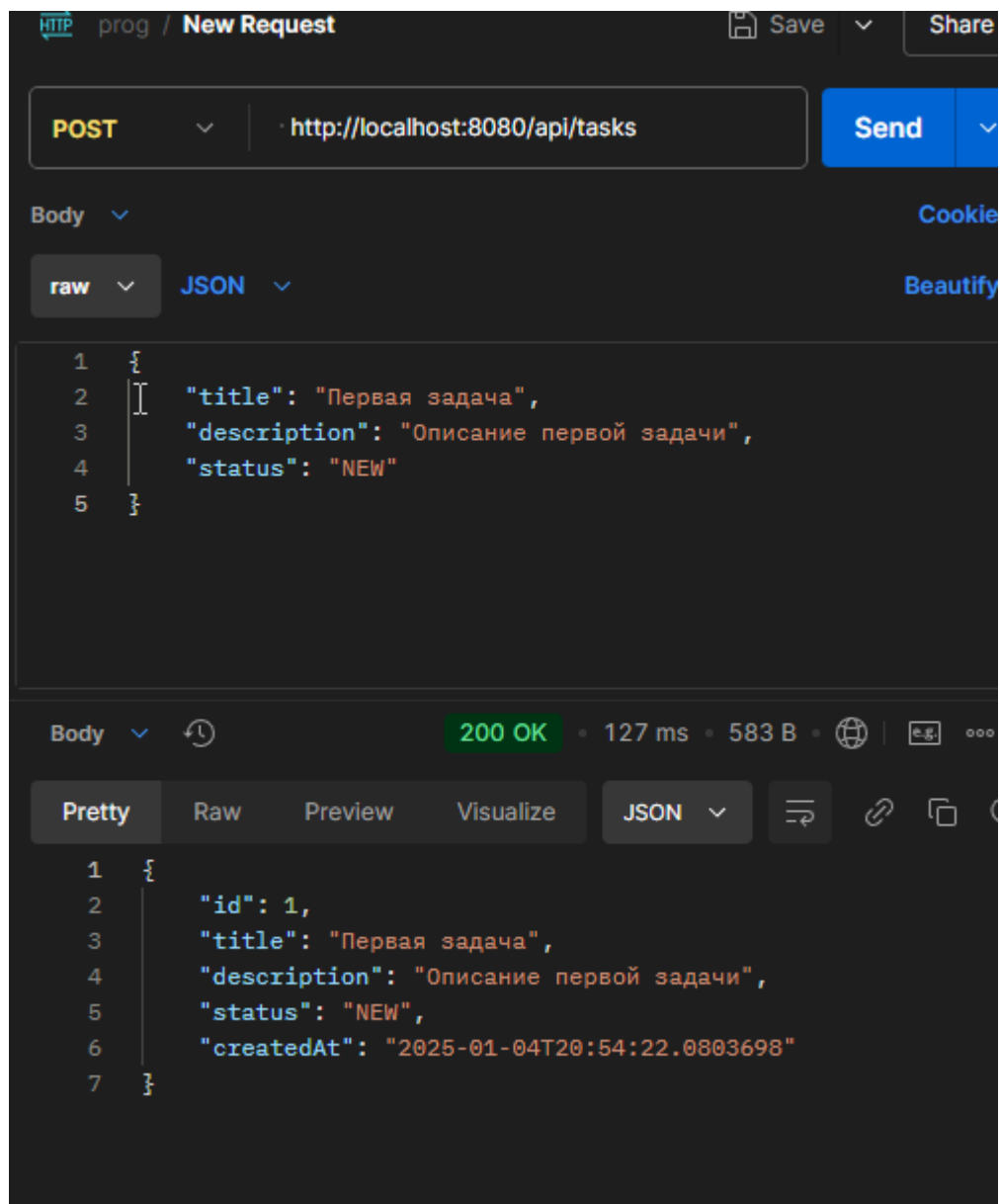
Auth Type

Basic Auth ▼

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username admin

Password admin123 ⚠



The screenshot shows the 'Body' tab of a REST client. The 'raw' tab is selected, displaying the request body as a JSON object. The 'JSON' tab is also visible. The response status is '200 OK' with a response time of 127 ms and a size of 583 B. The response body is also shown as a JSON object.

HTTP prog / New Request Save ▼ Share

POST ▼ http://localhost:8080/api/tasks Send ▼

Body ▼ Cookie Beautify

raw ▼ JSON ▼

```
1 {
2   "title": "Первая задача",
3   "description": "Описание первой задачи",
4   "status": "NEW"
5 }
```

Body ▼ 🕒 **200 OK** • 127 ms • 583 B • 🌐 | 🔗 📄 ⋮

Pretty Raw Preview Visualize JSON ▼ 🔄 🔗 📄 🔍

```
1 {
2   "id": 1,
3   "title": "Первая задача",
4   "description": "Описание первой задачи",
5   "status": "NEW",
6   "createdAt": "2025-01-04T20:54:22.0803698"
7 }
```

2. Проверка в H2

← ↻ localhost:8080 H2 Console

English ▾ Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▾

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:taskdb

User Name: sa

Password:

Connect Test Connection

← ↻ localhost:8080 H2 Console 90 %

🔊 🔍 ☒ Auto commit 🔧 🔍 Max rows: 1000 🔄 📄 Auto complete Off ▾ Auto select On ▾ ?

📁 jdbc:h2:mem:taskdb

📁 TASK

📁 INFORMATION_SCHEMA

👤 Users

📄 H2 2.2.224 (2023-09-17)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM TASK;

SELECT * FROM TASK;

ID	CREATED_AT	DESCRIPTION	STATUS	TITLE
1	2025-01-04 21:07:00.221665	Описание первой задачи	NEW	Первая задача

(1 row, 0 ms)

Edit

3. Получение списка задач (GET)

HTTP

prog / New Request

Save

Share

GET

http://localhost:8080/api/tasks

Send

Params

Auth

Headers (9)

Body

Scripts

Settings

Cookies

Auth Type

Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about [variables](#).

Username

admin

Password

admin123

Body

200 OK

147 ms

750 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

```
1  [
2    {
3      "id": 1,
4      "title": "Первая задача",
5      "description": "Описание первой задачи",
6      "status": "NEW",
7      "createdAt": "2025-01-04T21:07:00.221665"
8    },
9    {
10     "id": 2,
11     "title": "Первая задача",
12     "description": "Описание первой задачи",
13     "status": "NEW",
14     "createdAt": "2025-01-04T21:07:22.156837"
15   }
16 ]
```

4. Создание второй задачи (POST)

The screenshot displays the HTTP Client application interface. At the top, the URL bar shows 'http://localhost:8080/api/tasks' with a 'POST' method selected. The 'Send' button is visible. Below the URL bar, the 'Body' tab is active, showing a JSON payload in 'raw' mode:

```
1 {  
2   "title": "Вторая задача",  
3   "description": "Описание второй задачи",  
4   "status": "IN_PROGRESS"  
5 }
```

The response section at the bottom shows a '200 OK' status with a response time of 71 ms and a size of 596 B. The response body is displayed in 'Pretty' mode as a JSON object:

```
1 {  
2   "id": 3,  
3   "title": "Вторая задача",  
4   "description": "Описание второй задачи",  
5   "status": "IN_PROGRESS",  
6   "createdAt": "2025-01-04T21:07:52.557658"  
7 }
```

5. Обновление задачи (PUT)

The screenshot shows a REST client interface with a dark theme. At the top, it says "HTTP prog / New Request". There are "Save" and "Share" buttons. Below this, the HTTP method is set to "PUT" and the URL is "http://localhost:8080/api/tasks/1". A "Send" button is on the right. Below the URL bar, there are tabs for "Params", "Auth", "Headers (9)", "Body", "Scripts", and "Settings". The "Body" tab is selected. Below the tabs, there are buttons for "raw" and "JSON". The "JSON" button is selected. The main area shows the request body in JSON format:

```
1 {
2   "title": "Обновленная первая задача",
3   "description": "Новое описание первой задачи",
4   "status": "COMPLETED"
5 }
```

Below the request body, there is a status bar showing "Body", a refresh icon, "200 OK", "96 ms", "628 B", a globe icon, and "Save Response". Below the status bar, there are tabs for "Pretty", "Raw", "Preview", and "Visualize". The "Pretty" tab is selected. Below the tabs, there are buttons for "JSON" and a refresh icon. The main area shows the response body in JSON format:

```
1 {
2   "id": 1,
3   "title": "Обновленная первая задача",
4   "description": "Новое описание первой задачи",
5   "status": "COMPLETED",
6   "createdAt": "2025-01-04T21:07:00.221665"
7 }
```

6. Удаление задачи (DELETE)

prog / **New Request** Save Share

DELETE

http://localhost:8080/api/tasks/2

Send

Params

Auth

Headers (9)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body

200 OK

78 ms

388 B

Save Response

...

Pretty

Raw

Preview

Visualize

Text

1

HOST:0000

SQLZOO

50%

редактиро

0 Max rows: 1000 Auto complete Off Auto select On

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT * FROM TASK;

SELECT * FROM TASK;

ID	CREATED_AT	DESCRIPTION	STATUS	TITLE
1	2025-01-04 21:07:00.221665	Новое описание первой задачи	COMPLETED	Обновленная первая задача
3	2025-01-04 21:07:52.557658	Описание второй задачи	IN_PROGRESS	Вторая задача

(2 rows, 0 ms)

Edit

